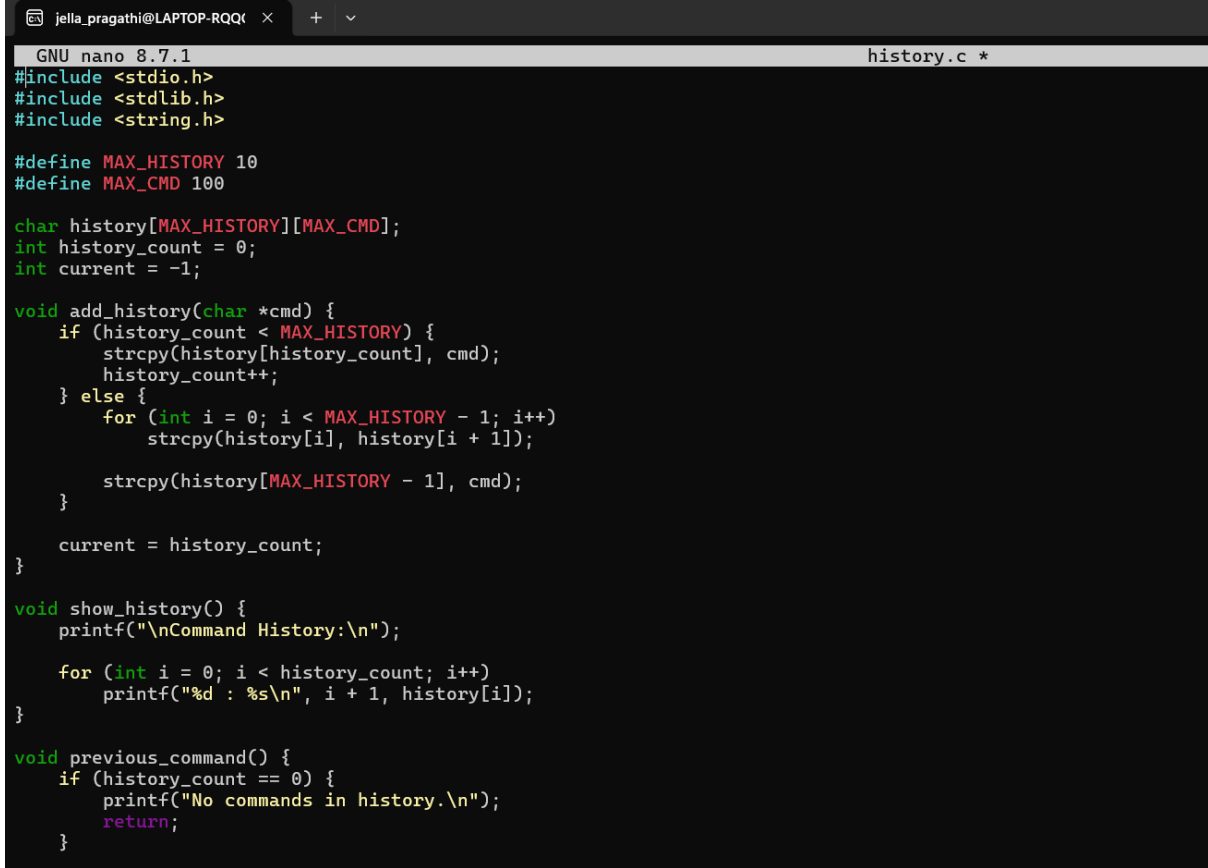


Skill WEEK 2

Session 3

Task 1:



```
GNU nano 8.7.1 history.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_CMD 100

char history[MAX_HISTORY][MAX_CMD];
int history_count = 0;
int current = -1;

void add_history(char *cmd) {
    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], cmd);
        history_count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
            strcpy(history[i], history[i + 1]);

        strcpy(history[MAX_HISTORY - 1], cmd);
    }

    current = history_count;
}

void show_history() {
    printf("\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
        printf("%d : %s\n", i + 1, history[i]);
}

void previous_command() {
    if (history_count == 0) {
        printf("No commands in history.\n");
        return;
    }
}
```

```
jella_pragathi@LAPTOP-RQQ... x + v
GNU nano 8.7.1 history.c *

    if (current > 0)
        current--;

    printf("Previous command: %s\n", history[current]);
}

void next_command() {
    if (history_count == 0) {
        printf("No commands in history.\n");
        return;
    }

    if (current < history_count - 1)
        current++;

    printf("Next command: %s\n", history[current]);
}

int main() {
    char command[MAX_CMD];

    printf("Command History System\n");

    while (1) {
        printf("\nEnter command: ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0) {
            show_history();
        }
        else if (strcmp(command, "prev") == 0) {
```

```
            previous_command(),
        }
        else if (strcmp(command, "next") == 0) {
            next_command();
        }
        else {
            add_history(command);
            printf("Command stored.\n");
        }
    }

    return 0;
}
```

```

jella_pragathi@LAPTOP-RQQGT555:~$ nano history.c
jella_pragathi@LAPTOP-RQQGT555:~$ ls
File1.txt  history.c  process.c  task2  task3.c  task5
File2.txt  ossp_project  task1  task2.c  task4  task5.c
OSSP      process  task1.c  task3  task4.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc history.c -o history
jella_pragathi@LAPTOP-RQQGT555:~$ ./history
Command History System

Enter command: ls
Command stored.

Enter command: pwd
Command stored.

Enter command: gcc test.c
Command stored.

Enter command: history

Command History:
1 : ls
2 : pwd
3 : gcc test.c

Enter command: prev
Previous command: gcc test.c

Enter command: next
Next command: gcc test.c

Enter command: exit
jella_pragathi@LAPTOP-RQQGT555:~$ |

```

Task 2:

```

GNU nano 8.7.1 buffer.c *
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void insert(Node **head, int value) {
    Node *newNode = malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = *head;
    *head = newNode;
}

void display(Node *head) {
    printf("Linked List: ");

    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }

    printf("NULL\n");
}

void freeList(Node *head) {
    Node *temp;

    while (head != NULL) {
        temp = head;

```

```
jella_pragathi@LAPTOP-RQQ... × + ~
GNU nano 8.7.1 buffer.c *

int main() {
    int size = 2;
    int count = 0;
    int value;

    int *buffer = malloc(size * sizeof(int));

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Dynamic Buffer Allocation\n");

    for (int i = 0; i < 5; i++) {
        printf("Enter value: ");
        scanf("%d", &value);

        if (count == size) {
            size *= 2;

            buffer = realloc(buffer, size * sizeof(int));

            if (buffer == NULL) {
                printf("Reallocation failed.\n");
                return 1;
            }

            printf("Buffer resized to %d elements.\n", size);
        }

        buffer[count++] = value;
    }

    printf("\nStored values: ");
}
```

```
        printf("Buffer resized to %d elements.\n", size);
    }

    buffer[count++] = value;
}

printf("\nStored values: ");

for (int i = 0; i < count; i++)
    printf("%d ", buffer[i]);

printf("\n");

Node *head = NULL;

for (int i = 0; i < count; i++)
    insert(&head, buffer[i]);

display(head);

free(buffer);
freeList(head);

printf("Memory released successfully.\n");

return 0;
}
```

```
jella_pragathi@LAPTOP-RQQG555:~$ nano buffer.c
jella_pragathi@LAPTOP-RQQG555:~$ gcc buffer.c -o buffer
./buffer
Dynamic Buffer Allocation
Enter value: 10
20
30
40
50Enter value: Enter value: Buffer resized to 4 elements.
Enter value: Enter value: 10
Buffer resized to 8 elements.

Stored values: 10 20 30 40 5010
Linked List: 5010 -> 40 -> 30 -> 20 -> 10 -> NULL
Memory released successfully.
jella_pragathi@LAPTOP-RQQG555:~$ ./buffer
Dynamic Buffer Allocation
Enter value: 10
Enter value: 20
Enter value: 30
Buffer resized to 4 elements.
Enter value: 40
Enter value: 50
Buffer resized to 8 elements.

Stored values: 10 20 30 40 50
Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> NULL
Memory released successfully.
jella_pragathi@LAPTOP-RQQG555:~$ valgrind --version
valgrind-3.26.0
jella_pragathi@LAPTOP-RQQG555:~$ valgrind --leak-check=full ./buffer
==1087== Memcheck, a memory error detector
==1087== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1087== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1087== Command: ./buffer
==1087==
Dynamic Buffer Allocation
Enter value: 10
Enter value: 20
Enter value: 30
Buffer resized to 4 elements.
Enter value: 40
Enter value: 50
Buffer resized to 8 elements.

Stored values: 10 20 30 40 50
Linked List: 50 -> 40 -> 30 -> 20 -> 10 -> NULL
Memory released successfully.
==1087==
==1087== HEAP SUMMARY:
==1087==    in use at exit: 0 bytes in 0 blocks
==1087== total heap usage: 10 allocs, 10 frees, 2,184 bytes allocated
==1087==
==1087== All heap blocks were freed -- no leaks are possible
==1087==
==1087== For lists of detected and suppressed errors, rerun with: -s
==1087== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
jella_pragathi@LAPTOP-RQQG555:~$ |
```

Session 4:

Task 1:

```
jella_pragathi@LAPTOP-RQQ... × + v
GNU nano 8.7.1 tokenizer.c *
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50
#define MAX_LENGTH 100

int main() {
    char input[MAX_LENGTH];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter an expression: ");
    fgets(input, MAX_LENGTH, stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count] = token;
        count++;

        token = strtok(NULL, " \t\n");
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i]);
    }

    printf("\nTotal tokens: %d\n", count);

    return 0;
}
```

```
jella_pragathi@LAPTOP-RQQGT555:~$ nano tokenizer.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc tokenizer.c -o tokenizer
jella_pragathi@LAPTOP-RQQGT555:~$ ./tokenizer
Enter an expression: add 10 + 20

Tokens:
Token 1: add
Token 2: 10
Token 3: +
Token 4: 20

Total tokens: 4
jella_pragathi@LAPTOP-RQQGT555:~$ |
```

Task 2:

```
jella_pragathi@LAPTOP-RQK... x + v
GNU nano 8.7.1 parser.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

int isNumber(char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i]))
            return 0;
    }
    return 1;
}

int main() {
    char input[100];
    char *tokens[10];
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    char *token = strtok(input, " \t");

    while (token != NULL && count < 10) {
        tokens[count++] = token;
        token = strtok(NULL, " \t");
    }

    if (count == 0) {
        printf("Error: Empty command.\n");
        return 0;
    }

    if (strcmp(tokens[0], "add") == 0) {
```

```
        if (strcmp(tokens[0], "add") == 0) {

            if (count != 3) {
                printf("Syntax Error: add requires two numbers.\n");
                return 0;
            }

            if (!isNumber(tokens[1]) || !isNumber(tokens[2])) {
                printf("Syntax Error: operands must be numbers.\n");
                return 0;
            }

            int a = atoi(tokens[1]);
            int b = atoi(tokens[2]);

            printf("Valid command.\n");
            printf("Result: %d\n", a + b);
        }

        else {
            printf("Error: Unknown command.\n");
        }

        return 0;
    }
```

```

jella_pragathi@LAPTOP-RQQT555:~$ nano parser.c
jella_pragathi@LAPTOP-RQQT555:~$ gcc parser.c -o parser
./parser
Enter command: add 10 20
Valid command.
Result: 30
jella_pragathi@LAPTOP-RQQT555:~$ add 10
Command 'add' not found, but there are 16 similar ones.
jella_pragathi@LAPTOP-RQQT555:~$ add hello 20
Command 'add' not found, but there are 16 similar ones.
jella_pragathi@LAPTOP-RQQT555:~$ ./parser
Enter command: add 10
Syntax Error: add requires two numbers.
jella_pragathi@LAPTOP-RQQT555:~$ ./parser
Enter command: add hello 20
Syntax Error: operands must be numbers.
jella_pragathi@LAPTOP-RQQT555:~$ ./parser
Enter command:
Error: Empty command.
jella_pragathi@LAPTOP-RQQT555:~$ |

```

Practice Questions:

Task 1:

```

jella_pragathi@LAPTOP-RQQT555:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
│   ├── builtin.o
│   ├── executor.o
│   ├── main.o
│   ├── parser.o
│   └── shell.o
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 24 files

```